

Group 1: Viktoriya Kurmisheva, Tien Manh Nguyen, Zaid Alayeddy

Mid Term - Exploratory Data Analysis

ITAI-1371

Viswanatha Rao

Reflection on Midterm Data Preprocessing Project

Before this project, our group practiced different preprocessing techniques in separate labs, but the midterm helped us see how they all connect in one complete workflow. The Patient Health Records dataset contained missing values, inconsistent text, invalid numerical entries, mixed date formats, and categorical data that a model could not use directly. Instead of using the same method for every problem, we had to decide which cleaning approach made the most sense for each type of data. This made the project more realistic because every preprocessing decision could affect what a future model learns (Tien, Viktoriya, and Zaid).

The workflow began with the required 70/30 train-test split before EDA (Tien). Records without a clear 0 or 1 Has_Disease value were excluded because supervised classification needs a known target, while stratification kept similar class proportions in both parts. Tien kept the testing data separate while the group used only training data for EDA and preprocessing decisions. This part of the project made data leakage easier to understand because using information from the test set during preparation could make the final evaluation less reliable (Tien).

The group also removed information that would not help the prediction model (Viktoriya). Patient_ID and Name identify individual patients rather than describe their health, so they added personal information without useful predictive value. Diagnosis_Code was removed because it could contain information too closely related to Has_Disease and make the prediction too easy for the model. This part of the project showed us that preprocessing is not only about fixing missing or incorrect values. It also means deciding which information should stay out of the model (Viktoriya).

Numerical cleaning covered BMI, Heart Rate, Cholesterol Level, Follow-Up, and visit dates (Tien). Units and text-based numbers were converted into usable values, non-exact cholesterol descriptions were treated as missing, and Follow-Up text such as two weeks was converted into days. For missing numerical values, Tien used the median from the training data because extreme values affect it less than the mean, and this allowed the group to keep more records. Visit dates were also separated into Visit_Year and Visit_Month so the date information could still be useful in numerical form (Tien).

Age and Blood Pressure needed extra corrections after closer review (Viktoriya). Known Age inconsistencies were corrected with Python, including twenty to 20, -5 to 5, and 250 to 25, before conversion to numerical form. Blood Pressure values written with /, over, or - were

changed into the same format and then separated into systolic and diastolic values. Any pair where the systolic value was less than or equal to the diastolic value was treated as invalid before filling in missing values. Fixing these values first prevented later blood-pressure features from being calculated from measurements that did not make sense (Viktoriya).

Categorical preprocessing focused on keeping the original meaning of the data while turning text into numbers (Viktoriya and Tien). Viktoriya cleaned categories in Gender, Diabetic, Smoker, Medications, and Notes. Blank medication entries became 0, while blank Notes entries became No_Notes. One-Hot Encoding then converted the categories into 0/1 columns without creating a false order between them. Tien standardized City and used training-data frequency encoding so City stayed in one numerical column, with frequencies rounded to four decimals. Viktoriya then balanced Has_Disease by random oversampling from 1,759 class-0 and 1,722 class-1 records to 1,759 in each class, which explains the duplicate rows created after balancing.

Feature engineering, normalization, and scaling prepared the cleaned numerical data for later model training (Zaid). Zaid created Pulse_Pressure, Mean_Arterial_Pressure, and Cholesterol_Age_Ratio from existing health measurements. He applied a log transformation to selected features with positive skew and used Min-Max Scaling on continuous numerical features that needed a common range. One-Hot Encoded columns stayed as 0/1 values, while Visit_Year and Visit_Month remained unchanged so they stayed easy to understand. Matplotlib graphs, printed samples, missing-value checks, class counts, and feature-range checks helped the group verify each stage (Tien, Viktoriya, and Zaid).

One challenge was keeping the 70/30 split correct while making sure the testing data stayed separate from EDA and preprocessing decisions (Tien). Several numerical columns also contained missing values, text, and inconsistent formats that needed careful cleaning (Tien). Age and Blood Pressure had unusual or invalid values that needed correction without removing useful patient records (Viktoriya). City was first encoded with One-Hot Encoding before we realized that frequency encoding would keep it in one numerical column and work better for our final dataset (Viktoriya). Deciding which numerical features needed normalization and scaling was another challenge because not every feature needed the same treatment (Zaid). Final range and distribution checks helped confirm that those transformations worked correctly (Zaid).

In conclusion, the final training dataset contained 3,518 records and 31 columns with no missing feature values, non-numerical predictors, or infinite values. Tien worked on the train-test split, numerical cleaning, date features, and City frequency encoding. Viktoriya handled identifiers, Age, Blood Pressure, categorical cleaning, One-Hot Encoding, and class balancing, while Zaid focused on feature engineering, normalization, and scaling. The group also created a separate Jupyter Notebook to show the main before-and-after changes. Most importantly, this project taught us that preprocessing is not just about cleaning data. Each decision needs a clear reason, and the results should be checked before the dataset is used for future machine learning work.